

dns-monitor manual

Adam Helešic

VUT FIT

November 2024

Contents

1	DNS	1
1.1	The problem	1
1.2	The solution	1
2	Application Description and Usage	1
2.1	Monitored Communication	1
2.1.1	Simplified	1
2.1.2	Verbose	1
2.2	Usage	2
3	Implementation	2
3.1	main.cpp	2
3.2	Configuration.cpp	2
3.3	CLParser.cpp	2
3.4	PacketCapture.cpp	3
3.5	PacketCaptureUtils.cpp	3
3.6	hashTable.cpp	3
3.7	DomainNameLogger.cpp, TranslationLogger.cpp	3
4	Testing	3

1 DNS

The internet as we know it would be much more tedious to use if DNS didn't exist. It would be like having to remember phone numbers or look them up in the golden pages whenever one wanted to access another device connected to the internet.

1.1 The problem

To access another device on the internet, it is necessary to know which device that is - a device is identified by an IP address which consists of 4 digits of values 0-255 (in the case of IPv4) or 32 hexadecimal digits (in the case of IPv6). A human could memorize a handful of these, but such a format is unnatural for the mind. We have a much easier time recalling information akin to language - whether existing words or made up words that are pronounceable. (We are of course not limited by pronounceability, but it does make it significantly easier to remember.) Hence why DNS was implemented. [2]

1.2 The solution

DNS (Domain Name System) is a system that can resolve domain names to IP addresses. When a domain name is used instead of an IP address, a query is made to one of the configured DNS servers and responses are sent back. [1]

DNS usually runs on port 53 using UDP. By capturing on that port, we can parse the obtained packets and gather information on the DNS communication in the network. [1]

2 Application Description and Usage

The main aim of this application is to monitor DNS communication on a given interface or to parse a pcap file and print the communication to the console. The application also provides the option to write all the domain names and or the domain name to IP translations to a file output.

2.1 Monitored Communication

The application has two types of output - simplified and verbose.

2.1.1 Simplified

Each DNS packet is condensed into a single line. The format is as follows:

```
<YYYY-MM-DD> <HH:MM:SS> <source IP> -> <destination IP> (<Q/R - query/response> <number of Question records>/<number of Answer records>/<number of Authority records>/<number of Additional records>)
```

Example simplified output:

```
2024-10-08 09:26:33 10.0.2.15 -> 192.168.1.1 (Q 1/0/0/1)
```

2.1.2 Verbose

Attention: records that are not of type A, AAAA, NS, MX, SOA, CNAME, or SRV are ignored in the resource records sections of verbose output.

The verbose output format is as follows:

Timestamp - date and time

SrcIP - source IPv4/6 address from the IP header

DstIP - destination IPv4/6 address from the IP header

SrcPort - source port: <TCP/UDP>/<port number>

DstPort - destination port: <TCP/UDP>/<port number>

Identifier - transaction ID (from DNS header)

Flags - values of the QR, OPCODE, AA, TC, RD, RA, AD, CD, RCODE flags (format: <uppercase flag name>=<value>)

Content of Question, Answer, Authority, Additional sections.

Example verbose output:

```
Timestamp: 2024-10-08 09:26:33
SrcIP: 192.168.1.1
DstIP: 10.0.2.15
SrcPort: UDP/53
DstPort: UDP/49884
Identifier: 0x9E19
Flags: QR=1, OPCODE=0, AA=0, TC=0, RD=1, RA=1, AD=0, CD=0, RCODE=0
```

```
[Question Section]
www.youtube.com. IN A
```

```
[Answer Section]
www.youtube.com. 300 IN CNAME youtube-ui.l.google.com
youtube-ui.l.google.com. 300 IN A 142.251.37.110
youtube-ui.l.google.com. 300 IN A 142.251.36.78
youtube-ui.l.google.com. 300 IN A 142.251.36.110
youtube-ui.l.google.com. 300 IN A 142.251.36.142
```

```
[Authority Section]
```

```
[Additional Section]
=====
```

2.2 Usage

```
./dns-monitor (-i <interface> | -p <pcapfile>) [-v] [-d <domainsfile>] [-t <translationsfile>]
```

Arguments:

- -i <interface> - the name of the interface to listen on, or
- -p <pcapfile> - the name of the .pcap file to parse;
- -v - "verbose" mode: complete information output of the DNS messages;
- -d <domainsfile> - the name of the domain names file;
- -t <translationsfile> - the name of the domain name to IP address translations file.

3 Implementation

3.1 main.cpp

The entry point of the application. Responsible for running the command line parser (**CLParser**) and packet capture.

3.2 Configuration.cpp

A class that holds the configuration of the application (as set up by the command line arguments) and initializes loggers if the optional switches (-d, -t) have been specified.

3.3 CLParser.cpp

A class that parses passed command line arguments using **getopts** and stores them in a **Configuration** instance. Validates that the switches are recognized and shows help if not. Validates that only an interface or only a pcap file was specified, or that at least one of them was specified. Validates that a pcap file exists and has the .pcap file extension. Initiates loggers through **Configuration**.

3.4 PacketCapture.cpp

Responsible for opening packet capture on the specified interface or processing the specified `pcap` file. Utilizes the `pcap` library.

Validates if the specified interface exists. Contains the `packet_handler` function from the `pcap` library that processes each packet.

The packet is an array of bytes in a set format - that means we are able to parse all the necessary information from it based on the byte positions. After the packet information is parsed and stored, we can print them to the output based on the mode (simplified or verbose).

3.5 PacketCaptureUtils.cpp

Contains utility functions for `PacketCapture`. These include things like transforming integer values to text (eg. resource record type and class), domain name extraction from the packet (including compressed names that are signified by the first 2 bits having the value of 1), extracting and printing common information from the DNS packet fields, etc.

3.6 hashTable.cpp

A basic hash table for storing domain names and domain name translations to check against duplicate entries.

3.7 DomainNameLogger.cpp, TranslationLogger.cpp

Classes for logging domain names and translations. They store each entry in a hash table and check for duplicates before logging to their specified file.

4 Testing

The main testing for this application was done using `Wireshark`. Two packet captures were done (one with IPv4, the other with IPv6 packets). The output of the application was then compared to `Wireshark`'s packet information.

Testing live capture on an interface was done using the `dig` command. The output of `dig` and the application matched on the tested domain names (`google.com`, `vut.cz`, `fit.vut.cz`).

References

- [1] Domain name system. *Wikipedia* (2024).
- [2] BASUMALLICK, C. 7 reasons why dns (domain name system) remains important. *Spiceworks Inc.* (2022).